Bài toán 2. Hệ thống lấy hàng truck và drone hoạt động song song, có ràng buộc time window, request của khách hàng đến ngẫu nhiên.

a) Hệ thống gồm:

- Depot 0.
- Tập K xe tải đồng nhất.
- Tập D drone đồng nhất.
- Horizon hoạt động: [0, T]
- Tập request C phát sinh ngẫu nhiên theo thời gian.

Mỗi request $i \in C$ có:

- Thời điểm xuất hiện: r_i.
- Khối lượng hàng: d_i.
- Cửa sổ thời gian phục vụ: $[e_i, l_i]$.
- Giới hạn waiting time: L_w (từ lúc lấy hàng đến lúc hàng về depot $\leq L_w$)
- Phân loại:
    - $i \in C_1$: chỉ truck phục vụ.
    - $i \in C_2$: truck hoặc drone.

b) Ràng buộc phương tiện

Truck

- Tải trọng tối đa: M_T.
- Mỗi hành trình: depot → (1 hoặc nhiều khách) → depot.
- Có thể thực hiện nhiều hành trình trong $[0, T]$.

Drone

- Tải trọng tối đa: M_D.
- Bán kính bay tối đa mỗi hành trình: L_D.
- Mỗi hành trình: depot → nhiều khách → depot.
- Xuất phát từ depot với pin đầy.

- Có thể thực hiện nhiều hành trình trong $[0, T]$.

c) Ràng buộc vận hành

Với mỗi request i:

1. Nếu được phục vụ:
    - Arrival time $a_i \leq l_i$.
    - Nếu $a_i < e_i$, phương tiện chờ đến e_i.
    - Pickup time: $p_i = \max(a_i, e_i)$.
    - Hàng phải được đưa về depot tại thời điểm q_i.
    - Waiting time:

$$W_i = q_i - p_i \leq L_w$$

2. Drone phải thỏa:
    - Tổng tải trên hành trình ≤ M_D
    - Tổng quãng bay ≤ L_D
3. Truck phải thỏa:
    - Tổng tải ≤ M_T

Chọn 1 trong các biến thể sau để giải:

1) Biến thể 1: đơn mục tiêu

Xây dựng động (online):

- Tập hành trình truck và drone.
- Quyết định với mỗi request i:
    - Phục vụ hay bỏ.
    - Nếu phục vụ: gán vào hành trình nào, thứ tự nào.

Gọi:

$$S_i = \begin{cases} 1 & \text{nếu request được phục vụ} \\ 0 & \text{nếu bị bỏ} \end{cases}$$

Hàm mục tiêu: tối thiểu hóa

$$\min J_{\text{drop}} = \alpha \cdot \left(Dist_T + Dist_D\right) + \gamma \sum_{i \in C} (1 - S_i)$$

với α, γ là các hệ số cố định cho trước;

$Dist_T$: tổng quãng đường truck; $Dist_D$: tổng quãng đường drone; $\sum(1 - S_i)$: số đơn bị bỏ.

(1) Greedy, (2) Rolling horizon, (3) Two-layer, hay (4) Meta-heuristic?

While $t \leq T$:

1. Nhảy tới event kế tiếp:
    - request mới đến
    - một truck/drone kết thúc tour/sortie (trở về depot)
2. Nếu event là request mới r, chạy thủ tục **GreedyAssign(r, t)** dưới đây.

2) Thủ tục GreedyAssign(r, t): chi tiết từng bước

Bước 2.1 — Xác định các lựa chọn hợp lệ theo loại request

- Nếu $c_r =$ TRUCK_ONLY⇒ chỉ xét truck.
- Nếu $c_r =$ FLEX⇒ xét cả truck và drone.

Bước 2.2 — Sinh tập ứng viên chèn (candidate insertions)

Tạo các "đề xuất chèn" dạng:

$$\text{cand} = (\text{vehicle } v, \text{tour } \tau, \text{position } (i, j), \Delta\text{cost})$$

Truck candidates

- Với mỗi truck k:
    - với mỗi tour τ của truck đó (kể cả option "tạo tour mới"):
        - thử chèn khách r vào mọi vị trí trong chuỗi (giữa 2 điểm liên tiếp)
        - mỗi vị trí cho ra 1 candidate

Drone candidates

- Tương tự: với mỗi drone m, mỗi sortie σ, thử mọi vị trí chèn, và option tạo sortie mới.

Chú ý triển khai: giới hạn số candidate bằng cách chỉ xét "tour gần nhất" hoặc "top-L vị trí tốt nhất" để chạy nhanh.

Bước 2.3 — Kiểm tra feasibility cho từng candidate (rất quan trọng)

Với mỗi candidate, cần **mô phỏng lại lịch thời gian** của tour sau khi chèn để kiểm tra ràng buộc, rồi kiểm tra từng ràng buộc có trong bài toán:

(a) Time window + waiting time của request

Giả sử tour sau chèn cho ra thời điểm đến khách r là A_r. Khi đó:

- Nếu $A_r < e_r$: xe được phép chờ đến e_r
  Pickup time $P_r = e_r$
- Nếu $e_r \leq A_r \leq l_r$:
  Pickup time $P_r = A_r$
- Nếu $A_r > l_r$: **infeasible**

Waiting time: Giả sử hàng được đưa về depot tại thời điểm q_r.

$$\text{wait}_r = q_r - A_r \leq L_w$$

và cần:

$$\text{wait}_r \leq W_r$$

(b) Ràng buộc "hàng phải về depot trước deadline"

Trong tour có điểm depot cuối, lấy thời điểm về depot sau khi hoàn tất tour là $Rtn(\tau)$. Cần:

$$Rtn(\tau) \leq T$$

(c) Ràng buộc tải trọng

- Truck: tổng tải trên tour $\leq Q_T$
- Drone: tổng tải trên sortie $\leq Q_D$

(d) Ràng buộc drone radius

Drone cần:

- total flight distance $\leq L_D$

(e) Ràng buộc horizon

- Tour phải hoàn thành trong $[0, T]$.

Nếu **bất kỳ ràng buộc nào fail** ⇒ loại candidate.

Bước 2.4 — Tính incremental cost cho candidate

Với candidate feasible, tính:

- Δd_{truck}: quãng đường truck tăng thêm nếu chèn r
- Δd_{drone}: quãng đường drone tăng thêm nếu chèn r
  Hàm đánh giá:

$$\Delta J = \pi_1 \Delta d_{\text{truck}} + \pi_2 \Delta d_{\text{drone}}$$

Có thể bổ sung greedy "nhìn xa" hơn chút, bằng cách cộng thêm penalty nếu slack time window bị giảm mạnh, ví dụ:

$$\Delta J' = \Delta J + \beta \cdot \Delta(\text{violation_risk})$$

Giải thích về slack time window: "**Slack time window**" (độ dư của cửa sổ thời gian) là cách đo mức "dễ thở" khi phục vụ một request: ta còn bao nhiêu thời gian dự phòng trước khi trễ hạn L_w. Nó rất hữu ích trong greedy online vì không chỉ nhìn Δquãng đường, mà còn tránh những quyết định làm lịch chạy "căng" quá, về sau dễ vỡ time window. Giả sử với một phương án chèn cụ thể, ta tính được **thời điểm đến** A_rtại khách r. Thời điểm bắt đầu phục vụ (pickup/service start) là:

$$S_r = \max(A_r, e_r)$$

Slack time window là: $\text{slack}_r = l_r - S_r$
Ý nghĩa:

- $\text{slack}_r < 0$: **vi phạm** time window (không feasible)
- $\text{slack}_r \approx 0$: vừa khít, chỉ cần trễ một chút là fail
- slack_rlớn: còn dư nhiều → lịch "an toàn" hơn

Tiếp theo, ta định nghĩa, **Slack của *một tour* sau khi chèn:** Khi chèn request r vào tour, không chỉ r bị ảnh hưởng, mà **các khách sau nó trong tour** cũng bị đẩy thời gian. Với tour sau chèn τ', ta cần tính slack cho **mọi khách** i trong tour theo công thức: $\text{slack}_i = l_i - S_i$
Từ đó, ta có thể đánh giá tour bằng một chỉ số, chọn 1 trong 3 loại sau:
(a) Min-slack (độ dư nhỏ nhất)

$$\text{SlackMin}(\tau') = \min_{i \in \tau'} \text{slack}_i$$

Đây là "nút cổ chai": chỉ cần nhiễu nhỏ là tour vỡ.
(b) Tổng slack / slack trung bình

$$\text{SlackSum}(\tau') = \sum_{i \in \tau'} \text{slack}_i$$

Dùng để đánh giá tour "thoáng" tổng thể.

(c) Decrease in slack do insertion

So sánh trước–sau:

$$\Delta\text{SlackMin} = \text{SlackMin}(\tau') - \text{SlackMin}(\tau)$$

Nếu ΔSlackMin âm lớn → việc chèn này làm tour căng hơn nhiều.

Tại sao slack quan trọng trong greedy online?

Nếu greedy chỉ tối thiểu hóa:

$$\Delta J = \pi_1 \Delta d_{\text{truck}} + \pi_2 \Delta d_{\text{drone}}$$

thì nó có thể chọn một chèn "rẻ" nhưng làm **SlackMin gần 0**.

Trong môi trường online:

- request tương lai chưa biết
- thời gian di chuyển có thể biến động
- có event gây trễ

⇒ tour "căng" sẽ **fail nhiều hơn** hoặc buộc reject các request sau dù đáng phục vụ.

Nói ngắn gọn: **Slack = độ robust** của lịch.

Cách đưa slack vào greedy (nhẹ, dễ implement): Weighted trade-off bằng "risk score"

Đặt một ngưỡng an toàn δ (ví dụ 5–10 phút):

$$\text{penalty}(\tau') = \beta \cdot \max\{0, \delta - \text{SlackMin}(\tau')\}$$

Score khi chèn thêm request mới: $\Delta J' = \Delta J + \text{penalty}(\tau')$

Ví dụ số nhanh để thấy được tầm quan trọng của slack

Request r: $[e_r, l_r] = [10,20]$

- Candidate A: đến $A_r = 12 \Rightarrow S_r = 12 \Rightarrow$ slack $= 20 - 12 = 8$
- Candidate B: đến $A_r = 19 \Rightarrow S_r = 19 \Rightarrow$ slack $= 1$

Nếu B có Δd nhỏ hơn một chút, greedy thuần distance có thể chọn B.

Nhưng chỉ cần trễ thêm 2 phút (kẹt xe / thêm khách mới) là B **vỡ TW**, còn A vẫn ổn.

Bước 2.5 — Chọn candidate tốt nhất

Có 2 tình huống:

(a) Bắt buộc reject request: không có phương án chèn để được lời giải feasible
tức là chèn đâu cũng vi phạm ít nhất 1 trong các ràng buộc của bài toán như time window / quá tải / drone vượt bán kính ⇒ reject.

(b) Tự chọn reject (optional): có feasible nhưng quá đắt
Nếu phục vụ làm tăng chi phí hơn ρ, thì theo objective tổng, bỏ sẽ tốt hơn.

Nói cách khác: ta so sánh

$$\min_{c \in \mathcal{C}_{feasible}} \Delta J'(c) \text{ với } \rho$$

c (tức là candidate) tương ứng với 1 vị trí chèn request vào lời giải hiện tại. $\mathcal{C}_{feasible}$ là tập các vị trí chèn request đang xét vào lời giải hiện tại để được 1 lời giải mới feasible. Mỗi vị trí c để chèn request, sẽ có 1 chi phí phục vụ request, đặt là $\Delta J'$ (c) như công thức đã đề cập ở phần trên.

- Nếu "chi phí phục vụ request rẻ hơn ρ" ⇒ phục vụ request
- Nếu "phục vụ đắt hơn" ⇒ bỏ

ρ chính là **giá trị ta sẵn sàng trả để "giữ chất lượng dịch vụ" (**chi phí bỏ đơn (tùy ta muốn ưu tiên service level hay distance)). ρ càng lớn thì hệ thống càng "cố phục vụ", càng nhỏ thì hệ thống càng dễ bỏ đơn.

Vậy chọn giá trị cho β, δ và ρ thế nào:

1) β: đặt cỡ tương đương "1 km quãng đường" để slack thiếu bị phạt rõ rệt

2) Chọn giá trị cho δ

Ta sử dụng EWMA (Exponentially Weighted Moving Average) là cách ước lượng "giá trị điển hình" của một đại lượng online (ở đây ta chọn là độ rộng time window $w_r = l_r - e_r$) mà không cần lưu toàn bộ lịch sử. Nó "nhớ" quá khứ nhưng giảm dần trọng số theo thời gian.

Khi request r đến, ta tính:

$$w_r = l_r - e_r$$

Ta duy trì một biến trạng thái m (ước lượng trung bình trượt). Mỗi lần có request mới, cập nhật giá trị của m theo công thức:

$$m \leftarrow (1 - \lambda)\, m + \lambda\, w_r$$

trong đó

- m: EWMA của w
- $\lambda \in (0,1]$: hệ số cập nhật (smoothing factor)

Ý nghĩa của công thức:

- m giữ lại $1 - \lambda$ phần của quá khứ
- "nhập" thêm λ phần thông tin mới

Có nhiều cách khởi tạo giá trị của m, để đơn giản, ta lấy ngay request đầu tiên làm mốc, do đó đặt khởi tạo: $m=w_1$

Còn giá trị của λ = 0.05. Chú ý:

- $\lambda = 0.05 \Rightarrow N_{\text{eff}} \approx 1/0.05 = 20$ request gần nhất ảnh hưởng mạnh
- $\lambda = 0.02 \Rightarrow N_{\text{eff}} \approx \frac{1}{0.02} = 50$ request gần nhất ảnh hưởng mạnh

Sau khi tính được giá trị m, ta đặt:

$$\delta(t) = \min\left(\delta_{\max}, \max\left(\delta_{\min}, \kappa\, m(t)\right)\right)$$

- κ thường 0.05–0.15 (nên chọn 0.1 ~ 10%)
- $\delta_{\min}$ và $\delta_{\max}$ thường để khoảng 10% độ rộng TW trung bình của các request trong bộ test. Ví dụ nếu trung bình độ rộng TW của các request trong bộ test là 30 – 120 phút, thì đặt $\delta_{\min} = 3$ (phút), $\delta_{\max}$ =12 (phút)

Ví dụ: Giả sử $\kappa = 0.1$. Nếu $m = 80$phút $\Rightarrow \delta = 8$phút.

Ví dụ: Chọn $\lambda = 0.2$. Có lần lượt 3 request đến có w_r: 60, 30, 90 phút.

- Khởi tạo $m_1 = 60$
- $m_2 = 0.8 \cdot 60 + 0.2 \cdot 30 = 48 + 6 = 54$
- $m_3 = 0.8 \cdot 54 + 0.2 \cdot 90 = 43.2 + 18 = 61.2$

Nhận thấy: giá trị mới 90 kéo m lên, nhưng không nhảy hẳn lên 90.

Sử dụng EWMA (Exponentially Weighted Moving Average) có nhược điểm là nếu thỉnh thoảng có request với giá trị w_r lớn hoặc nhỏ bất thường, sẽ có thể kéo m(trung bình trượt) lên/xuống mạnh, làm $\delta(t)$ thay đổi không mong muốn. [Ví dụ: Giả sử đa số request có độ rộng time window: $w_r = l_r - e_r$khoảng **30–60 phút;** Nhưng thỉnh thoảng có một request có: $w_r = 300$phút (rất rộng) **hoặc** $w_r = 2$phút (rất hẹp)]